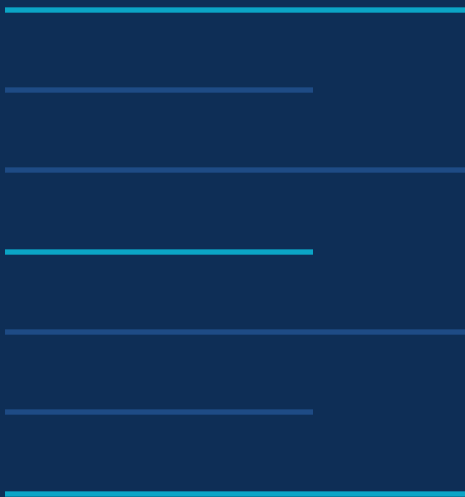


MÓDULO DE FORMACIÓN

Pruebas unitarias con GitHub Copilot

Generar, ampliar y mantener pruebas en Visual Studio Code con xUnit y los agentes Ask, Plan y Agent

Visual Studio Code · SDK de .NET · Kit de desarrollo de C#



Por qué probar con Copilot

Las pruebas unitarias garantizan la funcionalidad de los componentes individuales de un sistema. GitHub Copilot acelera su creación, reduce errores e identifica casos perimetrales y condiciones de límite, especialmente en bases de código grandes.

Copilot Chat en modo Agente

Modos Preguntar (Ask) y Plan para analizar y planificar antes de escribir código.

Sugerencias de texto fantasma

Completado en línea para crear y ampliar casos de prueba existentes.

xUnit como marco de pruebas

También compatibles NUnit y MSTest dentro del mismo flujo.

VS Code + Kit de desarrollo de C#

Hospedaje, detección, depuración y ejecución del proyecto de pruebas.

Temas del módulo

- 1 Hospedar y ejecutar pruebas con VS Code y el Kit de desarrollo de C#.
- 2 Generar pruebas desde Copilot Chat en modo Agente, explorando antes con el modo Preguntar.
- 3 Diseñar y automatizar flujos de varios archivos con los agentes Plan y Agent.
- 4 Ampliar la cobertura mediante sugerencias de texto fantasma.
- 5 Corregir pruebas con errores con ayuda de GitHub Copilot.
- 6 Desarrollar pruebas unitarias de extremo a extremo para una aplicación de C#.

Objetivos de aprendizaje

Entorno

Describir cómo VS Code, el SDK de .NET y el Kit de desarrollo de C# admiten pruebas en proyectos de C#.

Generación

Usar el modo Agente para generar pruebas de archivos y selecciones, y el modo Preguntar para explorar opciones.

Planificación

Diseñar la estrategia con el agente Plan y automatizar flujos de varios pasos con el agente Agent.

Cobertura y arreglo

Aplicar texto fantasma, el explorador de pruebas y `/fixTestFailure` para ampliar y reparar pruebas.

Requisito: suscripción activa

Suscripción a GitHub Copilot

- Plan gratuito de GitHub Copilot en una cuenta personal.
- Suscripción administrada por una organización o empresa.

Sugerencias con código público

Las actividades pueden generar sugerencias que coincidan con código público. En GitHub Enterprise Cloud esta configuración puede heredarse de la organización o empresa.

Si la cuenta bloquea las sugerencias que coinciden con código público, las actividades del módulo podrían no funcionar según lo previsto.

Compatibilidad de VS Code con pruebas

El proyecto necesita un marco de pruebas funcional y una forma de ejecutarlas dentro de Visual Studio Code.



Funcionalidades del Kit de desarrollo de C#

- Explorador de pruebas: vista de árbol con todos los casos del área de trabajo.
- Ejecutar o depurar: botones de reproducción junto a cada clase y método.
- Resultados en el editor y en el explorador, con enlaces a los seguimientos de pila.
- Comandos como Test: Run All Tests en la paleta de comandos.
- Configuración de detección y ejecución con las opciones Testing.
- Marcos admitidos: xUnit, NUnit y MSTest.

Creación de proyectos de prueba

Ejecutar **.NET: Nuevo Project...** · Ctrl+Shift+P (Windows/Linux) · Cmd+Shift+P (macOS) · Vista → Paleta de comandos

xUnit

xUnit Test Project

- Microsoft.NET.Test.Sdk
- xUnit
- xunit.runner.visualstudio
- coverlet.collector

NUnit

NUnit3 Test Project

- Microsoft.NET.Test.Sdk
- NUnit
- NUnit3TestAdapter

MSTest

MSTest Test Project

- Microsoft.NET.Test.Sdk
- MSTest.TestAdapter
- MSTest.TestFramework
- coverlet.collector

```
dotnet add [ruta del csproj de pruebas] reference [ruta del csproj del proyecto a probar]
```

Ejecución y administración de pruebas

Ejecutar o depurar desde el editor

Botón de reproducción junto a una clase o método de prueba.

Usar el Explorador de pruebas

Ejecutar o depurar pruebas individuales, grupos o todo el conjunto.

Consultar resultados

Estados de aprobación, errores y duración de cada prueba.

Configurar las pruebas

Opciones Testing: ejecución automática al guardar y formato de resultados.

Comandos de prueba disponibles

Test: Run All Tests

Test: Debug Failed Tests

Test: Show Output

Flujo de trabajo en tres fases



Las unidades restantes se centran en las herramientas de Copilot para generar, ejecutar y mantener las pruebas.

La vista Chat de Visual Studio Code

Lugar principal para configurar el marco, generar pruebas para un archivo o selección, refinarlas y ejecutarlas iterando sobre los errores en una misma sesión. Se abre con `Ctrl+Alt+I` (`Cmd+Alt+I` en macOS) o desde el icono de Copilot → Toggle Chat.

Destino del agente

Determina dónde se ejecuta el agente. Local permite trabajar en el editor con acceso al área de trabajo, herramientas y modelos.

Agente

Define el rol de la IA. Los agentes locales integrados son Ask, Plan y Agent.

Nivel de permiso

Aprobaciones predeterminadas ·
Omisión de aprobaciones · Autopilot.

Recomendado: Agente + aprobaciones predeterminadas — el Agente edita archivos y ejecuta pruebas, y las aprobaciones mantienen informado al usuario.

Modo Preguntar y configuración inicial

Modo Preguntar (Ask)

Analiza y planifica sin modificar archivos ni invocar herramientas.

- Comparar casos de prueba para métodos complejos.
- Identificar casos perimetrales y condiciones de límite.
- Recomendaciones de marcos o estilos de aserción.
- Ver ejemplos de pruebas sin escribirlos en disco.

Flujo: abrir Chat → seleccionar Preguntar → adjuntar el archivo o selección → preguntar → cambiar a Agente para generar.

Configuración con /setupTests

Configura un marco de pruebas cuando el proyecto todavía no lo tiene.

- Recomendar un marco de pruebas.
- Instalar los paquetes necesarios.
- Crear el proyecto de pruebas.
- Recomendar extensiones de pruebas para VS Code.

Flujo: abrir Chat → seleccionar Agente → ejecutar /setupTests → confirmar herramientas y comandos.

Generar pruebas con `/tests`

El comando `/tests` genera pruebas unitarias para el código activo en el editor. En modo Agente se escriben directamente en un archivo adecuado: Copilot detecta el marco existente y el estilo de codificación del proyecto.

Para un archivo completo

- Abrir el archivo de código y Copilot Chat en modo Agente.
- Ejecutar `/tests` con instrucciones adicionales.
- Confirmar las herramientas de lectura, escritura y ejecución.
- Revisar los cambios y seleccionar Mantener o Deshacer.

Para un método o bloque

- Abrir el archivo y seleccionar el método o bloque.
- Ejecutar `/tests` indicando que se use la selección.
- Revisar y mantener o descartar los cambios.

También sirve el lenguaje natural: "crear pruebas unitarias para `CalculateDiscount`, con casos límite para valores negativos y cero", incluyendo la ejecución posterior en el mismo mensaje.

Contexto, revisión y personalización

Agregar contexto

- Adjuntar archivos, carpetas, símbolos o la selección actual.
- Arrastrar archivos desde el Explorador o las pestañas.
- Menciones `#selection` y `#codebase`.
- Archivos Markdown con convenciones de prueba.

Revisar y refinar

- Revisar la diferencia resaltada en el editor.
- Mantener o deshacer los cambios.
- Compilar y ejecutar desde el explorador o el terminal.
- Iterar con mensajes posteriores.

Personalizar

- Marcos preferidos, como xUnit o NUnit.
- Convenciones de nombres de clases y métodos.
- Estructura Arrange-Act-Assert.
- Pruebas parametrizadas para valores límite.

Las instrucciones personalizadas viven en archivos `*.instructions.md` y el campo `applyTo: tests/**` limita su alcance; compartirlas por control de código fuente unifica al equipo.

Comparación de agentes

Agente	Adecuado para	Uso típico en pruebas unitarias
Preguntar (Ask)	Análisis de solo lectura y preguntas sobre el código	Explorar casos perimetrales, opciones de marco y ejemplos antes de escribir código.
Plan	Planes de implementación estructurados	Diseñar estrategias de pruebas de varios archivos para revisar antes de implementar.
Agent	Flujos autónomos y codificación de varios archivos	Generar pruebas, ejecutarlas y corregir errores mediante iteraciones.

Solicitudes Premium

En modo Agent, Copilot puede realizar varias solicitudes Premium para una sola tarea — tanto para el mensaje inicial como para las acciones posteriores. El total depende de la complejidad de la tarea, la cantidad de pasos y el modelo seleccionado.

Planear con Plan, ejecutar con Agent

Agente Plan

Investiga la tarea, formula preguntas aclaratorias y propone un plan paso a paso antes de escribir código.

- Resumen de alto nivel y desglose de pasos.
- Pasos de comprobación para ejecutar las pruebas.
- Decisiones documentadas y revisables.
- Útil con varios archivos, nuevos accesorios o convenciones de equipo.

Agente Agent

Automatiza tareas de varios pasos en todo el espacio de trabajo, con confirmación de herramientas y comandos.

- Preparar mediante scaffolding un proyecto de pruebas.
- Crear archivos de prueba y ejecutarlos.
- Generar informes de resultados.
- Corregir los problemas encontrados durante la ejecución.

Plan primero — Ambigüedad, varios archivos o convenciones por confirmar.

Agent directamente — Tarea bien definida, sin planificación intermedia.

Plan → Agent — Plan revisable más implementación autónoma.

Ampliar cobertura y corregir errores

Texto fantasma

Finalización de código que aparece mientras se escribe, basada en los patrones existentes del archivo de pruebas.

- Colocar el cursor tras el último caso y pulsar Enter.
- Empezar un método o escribir un comentario con el escenario.
- Tab para aceptar la sugerencia, Esc para descartarla.
- Funciona mejor con Arrange-Act-Assert ya presente.

Corrección de pruebas con errores

Tres caminos según el contexto disponible y el número de fallos.

- Explorador de pruebas: "Corregir error de prueba" abre Chat con la prueba y su salida.
- `/fixTestFailure`: iniciar desde Chat adjuntando archivos o salida del terminal.
- Agent: ejecuta, diagnostica, corrige y vuelve a ejecutar automáticamente.

La corrección puede afectar al código de la aplicación, al código de prueba o a ambos. Volver a ejecutar la prueba para confirmarla.

Elección de la herramienta

Herramienta	Uso recomendado
Texto fantasma	Agregar casos a un archivo de pruebas existente que ya contiene el patrón deseado.
Corregir error en el Explorador de pruebas	Corregir rápidamente una única prueba con errores.
/fixTestFailure	Adjuntar contexto adicional o resolver varios fallos desde Chat.
Agent	Ejecutar pruebas, diagnosticar errores y aplicar correcciones en varios archivos durante una sesión.

Chat, Plan y Agent generan las pruebas iniciales; el texto fantasma amplía la cobertura y las herramientas de corrección mantienen el conjunto funcionando a medida que evoluciona el código.

IMPORTANTE

La revisión manual sigue siendo necesaria

Las pruebas generadas pueden no cubrir todos los escenarios. La revisión manual y la revisión de código siguen siendo imprescindibles para garantizar la calidad del conjunto de pruebas.

Configurar el entorno

Generar con Chat, Plan y Agent

Ampliar y mantener las pruebas